

《面向对象程序设计实践》

课程论文

题目：	通讯录管理程序
专业：	软件工程
班级：	24 级 5 班
小组成员 1：	202425220511 龚慧怡
小组成员 2：	202425220530 周艳雯
指导老师：	彭红星
提交时间：	2026 年 5 月 18 日

华南农业大学 数学与信息学院 软件学院

目录

1 系统分析

- 1.1 问题描述
- 1.2 系统功能分析
- 1.3 开发平台及工具介绍
- 1.4 可行性与约束分析

2 系统设计

- 2.1 系统总体结构设计
- 2.2 系统各个类及类之间关系设计
- 2.3 数据存储设计
- 2.4 界面设计

3 系统实现

- 3.1 程序启动与初始化
- 3.2 联系人管理实现
- 3.3 查询和拼音搜索实现
- 3.4 CSV 与 vCard 导入导出实现
- 3.5 数据持久化实现

4 系统测试

- 4.1 测试环境
- 4.2 模块测试
- 4.3 系统功能完整测试
- 4.4 测试结论

5 系统运行界面

- 5.1 主界面
- 5.2 联系人编辑界面
- 5.3 联系人详情界面

6 总结

- 6.1 小组总结
- 6.2 成员个人体会

参考资料

附录

1 系统分析

1.1 问题描述

根据《附件 1-2026 春-面向对象程序设计实践-可选题目.docx》中题目 3 “通讯录管理程序”的要求，本实验需要完成一个单机版通讯录应用程序。系统的核心目标是使用 Java 图形用户界面技术实现联系人资料的管理，并通过文件系统保存通讯录数据，使用户能够在本机完成联系人新增、修改、删除、分组、查询、搜索以及通讯录数据导入导出等操作。

题目要求每个联系人至少包括姓名、电话、手机、即时通信工具及号码、电子邮箱、个人主页、生日、像片、工作单位、家庭地址、邮编、所属组和备注等字段。当前 contacts 项目中 Contact 类已经覆盖这些字段，并使用 UUID 作为内部唯一标识；AddressBook 类负责维护联系人集合和分组集合，形成了较清晰的对象模型。

从使用场景看，通讯录程序需要同时满足“快速查找”和“稳定保存”两个要求。用户希望在联系人数量增加后仍能通过姓名、电话、手机、拼音首字母等方式快速定位联系人，也希望分组调整、字段修改和导入导出后数据不会丢失。因此，本系统将界面交互、业务模型和文件读写分离，降低后续扩展难度。

1.2 系统功能分析

结合题目要求和 contacts 源码，系统功能可以划分为联系人管理、分组管理、查询搜索、字段显示控制、数据导入导出和本地持久化六个部分。各部分之间既相互独立，又通过 AddressBook 统一维护业务状态。

功能模块	功能说明	代码依据
联系人管理	新增、编辑、删除联系人；联系人字段完整覆盖题目要求。	ContactDialog、MainFrame、AddressBook
分组管理	新增分组、删除分组；删除分组时保留联系人，仅移出该分组。	AddressBook.addGroup/removeGroup
分组调整	对勾选或当前选中联系人执行加入分组、移出分组操作。	MainFrame.groupOperation
查询联系人	左侧分组列表选择全部、未分组或指定分组，右侧列表同步显示。	AddressBook.queryContacts
搜索联系人	支持姓名、电话、手机、姓名全拼和声母搜索，并按姓名排序。	PinyinUtils、matchesKeyword
字段显示	用户可以选择表格中显示的联系人属性，满足题目“可调整显示属性”的要求。	ContactTableModel、chooseColumns
导入导出	支持 CSV 和 vCard 文件的导入导出；导入时进行重复联系人提示。	CsvService、VCardService
数据存储	以 XML 文件保存分组与联系人，启动时加载，修改后写回。	AddressBookStorage、data/address-book.xml

1.3 开发平台及工具介绍

项目	说明
开发语言	Java，使用 Java SE 标准库完成 Swing 界面、集合处理、XML 解析和文件读写。
图形界面	Swing。主窗口继承 JFrame，编辑窗口继承 JDialog，表格模型继承 AbstractTableModel。
运行环境	本次检查环境为 Java 25.0.1 LTS；源码使用的语法适合 Java 17 及以上版本。
开发工具	项目包含 IntelliJ IDEA 模块文件 contacts.iml 和 .idea 配置，可直接作

	为 Java 模块打开。
数据格式	主数据文件为 XML，同时支持 CSV、vCard 作为交换格式。
第三方依赖	核心程序未引入外部库，便于课程设计提交和跨机器运行。

1.4 可行性与约束分析

技术可行性方面，Swing 可以满足课程设计对桌面图形界面的要求，JDK 自带的 DOM、Transformer、Files 等 API 可以完成 XML、CSV、vCard 等文件处理。本系统没有依赖数据库或网络服务，部署时只需要 JDK 环境和项目源码即可。

数据可行性方面，XML 文件结构直观，适合保存课程设计规模的联系人数据。联系人较多时，线性查询会带来一定性能压力，但在单机通讯录常见数据量下可以满足需求。后续若需要扩展到大量数据，可以在 AddressBook 中增加索引或迁移到 SQLite 等轻量级数据库。

交互约束方面，程序面向单机用户，未处理多进程同时写入同一 XML 文件的并发问题；导入 vCard 时兼容常见字段，但不同手机厂商可能会扩展私有字段，本系统将部分未知 X- 字段保存到备注中，以尽量减少信息丢失。

2 系统设计

2.1 系统总体结构设计

系统采用分层设计思路：界面层负责用户交互，业务模型层负责联系人和分组规则，数据访问层负责文件读写和格式转换，工具层提供拼音处理等辅助能力。这样的结构使界面代码不直接操作 XML 细节，导入导出服务也不需要了解 Swing 控件状态。

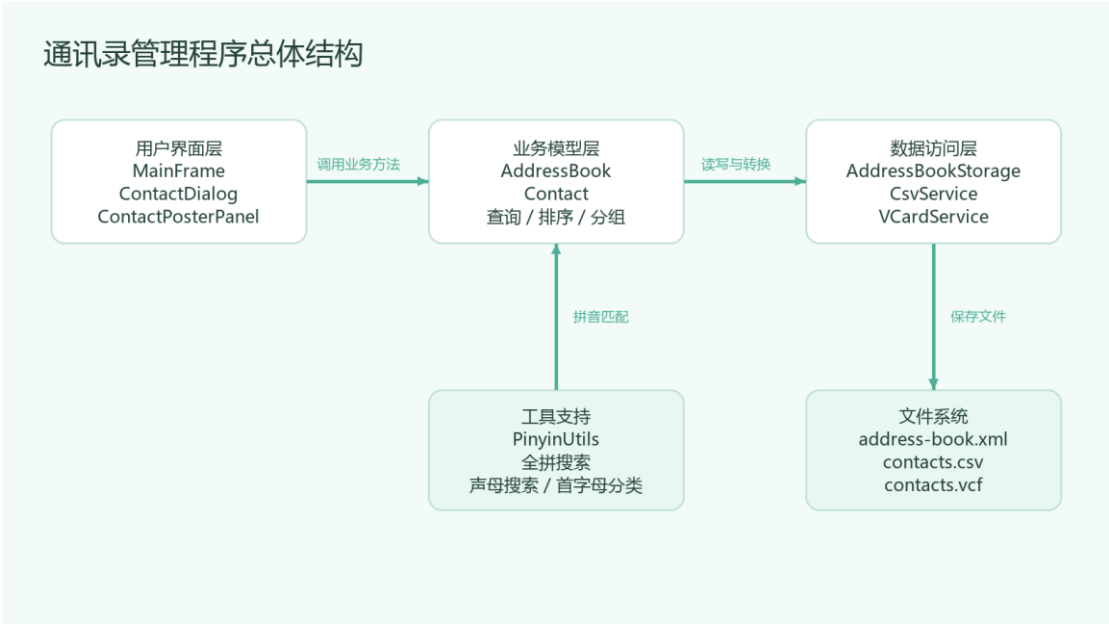


图 2-1 系统总体结构图

MainFrame 在程序运行时连接各层：它根据用户操作调用 AddressBook 完成业务变更，随后调用 AddressBookStorage.save 写回 XML 文件，并刷新分组列表、联系人表格、状态栏和详情面板。

2.2 系统各个类及类之间关系设计

主要类及职责关系

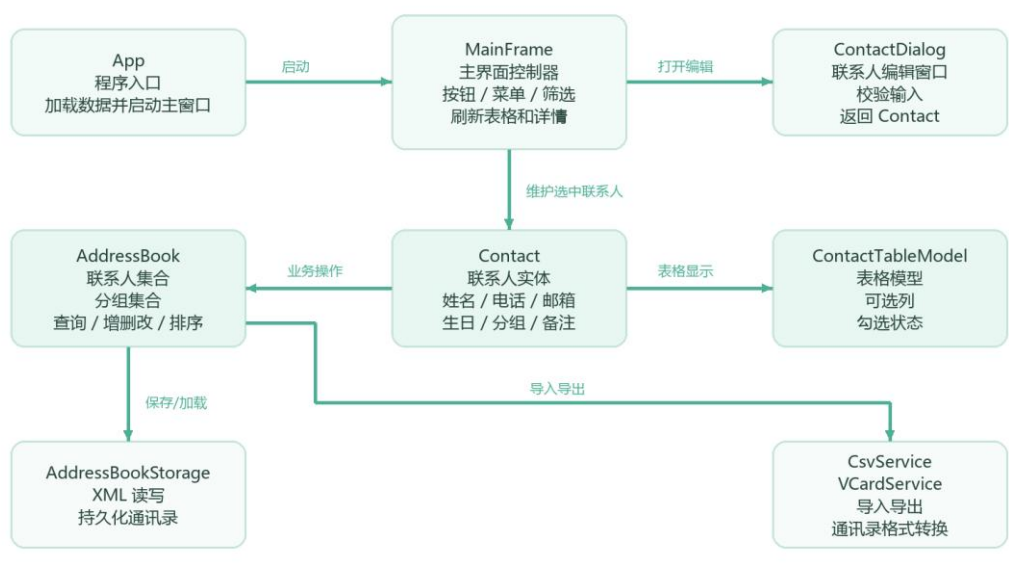


图 2-2 主要类职责关系图

类名	主要职责
App	程序入口，设置 Nimbus 外观，加载 data/address-book.xml；若数据为空则生成示例联系人。
Contact	联系人实体类，封装姓名、电话、手机、邮箱、生日、照片、分组、备注等字段，并提供 copy 方法避免外部直接修改内部对象。
AddressBook	业务模型类，维护联系人列表和分组集合，实现增删改、分组调整、按条件查询、拼音搜索和排序。
AddressBookStorage	XML 存储类，负责将 AddressBook 与 address-book.xml 之间互相转换。
CsvService	CSV 导入导出类，按题目要求字段导出 UTF-8 CSV，并解析带引号的 CSV 行。
VCardService	vCard 导入导出类，支持 vCard 3.0 导出，并兼容常见 vCard 字段、折行、生日、地址、分组和扩展字段。
MainFrame	主窗口控制类，组织分组列表、联系人表格、搜索框、操作按钮、右键菜单、导入导出和状态栏。
ContactDialog	联系人新增/编辑窗口，提供字段输入、生日格式校验、照片路径预览和自定义分组选择。
ContactTableModel	表格模型，管理可见列、勾选联系人以及不同字段的显示值。
ContactPosterPanel	联系人详情面板，以卡片方式展示头像、联系方式、基础资料、分组和备注。
PinyinUtils	拼音工具类，提供姓名转全拼、首字母和排序分桶。

2.3 数据存储设计

本系统主存储文件位于 contacts/data/address-book.xml。根元素 addressBook 下分为 groups 和 contacts 两部分：groups 保存全局分组；contacts 下每个 contact 元素保存联系人字段，并通过 contactGroups 子节点保存该联系人所属分组。

XML 存储结构示例

```
<addressBook>
  <groups>
    <group>家人</group>
    <group>朋友</group>
    <group>同事</group>
  </groups>
  <contacts>
    <contact>
      <name>张三</name>
      <phone>13800138000</phone>
      <email>zhangsan@example.com</email>
      <birthday>1990-01-01</birthday>
      <photo>photo.jpg</photo>
      <groups>
        <group>家人</group>
      </groups>
    </contact>
  </contacts>
</addressBook>
```

```
</groups>
<contacts>
  <contact id="...">
    <name>张三</name>
    <phone>010-62310001</phone>
    <mobile>13800001111</mobile>
    <email>zhangsan@example.com</email>
    <contactGroups>
      <group>朋友</group>
      <group>同事</group>
    </contactGroups>
  </contact>
</contacts>
</addressBook>
```

该结构既保留了全局分组顺序，也允许联系人属于多个分组。当删除某个分组时，`AddressBook.removeGroup` 会删除全局分组名，同时遍历联系人并移除该分组引用，保证题目要求的“删除分组不删除联系人”。

2.4 界面设计

界面采用左右分区布局。左侧为分组列表，显示全部联系人、未分组以及用户自定义分组，并在每一项右侧显示人数；中间为联系人列表，支持勾选、多列展示、右键菜单和字段显示控制；右侧为详情面板，选中联系人后展示头像、联系方式、基础资料、所属分组和备注。

顶部提供搜索框和核心操作按钮，包括新增联系人、编辑、删除、新建分组、删除分组、加入分组、移出分组等。搜索框监听 `DocumentEvent`，输入变化后立即刷新联系人列表。底部状态栏显示当前分组、列表人数、勾选人数以及搜索关键词，帮助用户判断当前操作范围。

联系人编辑窗口按“基本信息、联系方式、照片、分组、地址、备注”分区组织字段。窗口中姓名为必填项，生日使用 `yyyy-MM-dd` 格式校验，照片字段支持本地路径预览，自定义分组可以在编辑时新增并随联系人保存。

3 系统实现

3.1 程序启动与初始化

程序入口为 `contacts.App`。主线程通过 `SwingUtilities.invokeLater` 进入事件派发线程，先设置 Nimbus 外观，再创建 `AddressBookStorage` 读取 `data/address-book.xml`。如果首次运行时没有联系人，程序会写入示例联系人，便于用户立即看到系统效果。

程序启动关键代码

```
File dataDir = new File("data");
File dataFile = new File(dataDir, "address-book.xml");
AddressBookStorage storage = new AddressBookStorage(dataFile);
AddressBook addressBook = storage.load();

if (addressBook.getContacts().isEmpty()) {
    seedSampleContacts(addressBook);
    storage.save(addressBook);
}

MainFrame frame = new MainFrame(addressBook, storage);
frame.setVisible(true);
```

3.2 联系人管理实现

联系人实体使用 `Contact` 类封装。字符串字段在 setter 中统一执行 null 转空串和 trim 处理，避免界面输入为空时出现空指针。`Contact.copy` 用于复制联系人对象，使 `AddressBook` 内部存储的是独立副本，减少界面层直接修改模型对象造成的状态混乱。

联系人与分组管理核心代码

```
public void addContact(Contact contact) {
    Contact stored = contact.copy();
    contacts.add(stored);
    groups.addAll(stored.getGroups());
}

public void updateContact(Contact updatedContact) {
    for (int i = 0; i < contacts.size(); i++) {
        if (contacts.get(i).getId().equals(updatedContact.getId())) {
            contacts.set(i, updatedContact.copy());
            groups.addAll(updatedContact.getGroups());
            return;
        }
    }
    addContact(updatedContact);
}

public void removeGroup(String groupName) {
    String normalized = normalize(groupName);
    groups.remove(normalized);
    contacts.forEach(contact -> contact.getGroups().remove(normalized));
}
```


MainFrame 中新增和编辑共用 ContactDialog。新增时传入 null，编辑时传入当前选中联系人；弹窗保存后返回 Contact 对象，再由 AddressBook.updateContact 统一插入或覆盖。删除联系人时优先处理勾选项，没有勾选项时处理当前选中行，并在执行前弹出确认提示。

3.3 查询和拼音搜索实现

查询功能由 AddressBook.queryContacts 完成。方法先根据左侧分组过滤联系人，再根据搜索框关键词过滤，最后使用 Collator 按中文姓名排序。关键词匹配范围包括姓名、电话、手机、姓名全拼和姓名声母。

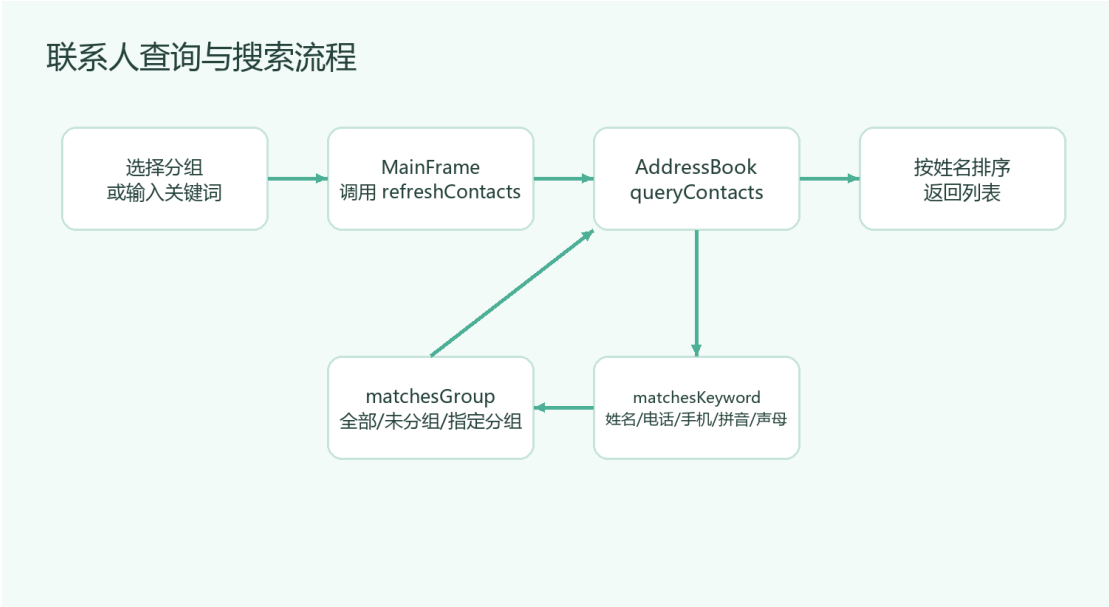


图 3-1 查询与搜索流程图

查询与拼音搜索关键代码

```
public List<Contact> queryContacts(String groupFilter, String searchText) {
    String group = groupFilter == null ? ALL_CONTACTS : groupFilter;
    String keyword = normalize(searchText).toLowerCase(Locale.ROOT);
    return contacts.stream()
        .filter(contact -> matchesGroup(contact, group))
        .filter(contact -> matchesKeyword(contact, keyword))
        .sorted(CONTACT_COMPARATOR)
        .collect(Collectors.toList());
}

private boolean matchesKeyword(Contact contact, String keyword) {
    if (keyword.isEmpty()) {
        return true;
    }
    String fullPinyin =
PinyinUtils.toPinyin(contact.getName()).toLowerCase(Locale.ROOT);
    String initials =
PinyinUtils.toInitials(contact.getName()).toLowerCase(Locale.ROOT);
    return contact.getName().toLowerCase(Locale.ROOT).contains(keyword)
        || contact.getPhone().toLowerCase(Locale.ROOT).contains(keyword)
        || contact.getMobile().toLowerCase(Locale.ROOT).contains(keyword)
        || fullPinyin.contains(keyword)
```

```

        || initials.contains(keyword);
    }

```

PinyinUtils 中包含常见汉字拼音区间表，并对“重、厦、沈、曾、解、单”等多音姓进行覆盖处理。表格模型中“分类”列调用 sortBucket，能够按姓名首字母将联系人分桶显示。

3.4 CSV 与 vCard 导入导出实现

CSV 导出时按题目字段顺序输出表头，并对所有字段加引号，字段中的引号使用双引号转义。导入时按逗号解析，同时处理引号包裹内容，避免家庭地址、备注中包含逗号时被错误拆分。vCard 导出采用 VERSION:3.0，写入 FN、N、TEL、EMAIL、URL、BDAY、ORG、ADR、PHOTO、CATEGORIES、NOTE 等字段。

CSV / vCard 导入导出流程

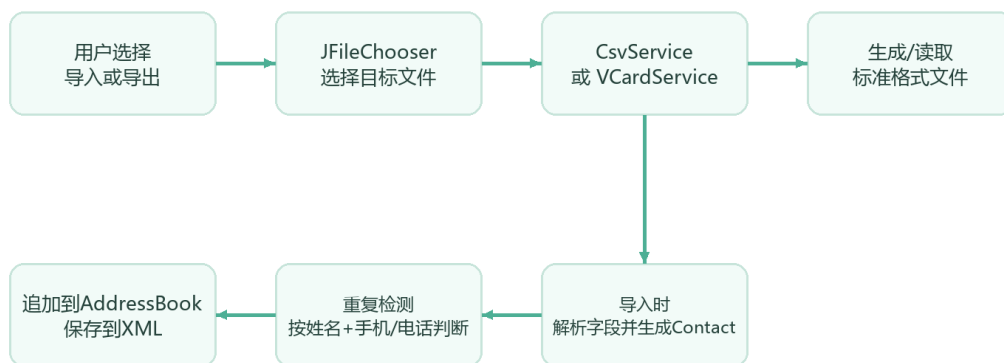


图 3-2 CSV/vCard 导入导出流程图

导入导出与重复联系人处理片段

```

if ("csv".equals(type)) {
    csvService.exportContacts(dataToExport, fileChooser.getSelectedFile());
} else {
    vCardService.exportContacts(dataToExport, fileChooser.getSelectedFile());
}

List<Contact> imported = "csv".equals(type)
    ? csvService.importContacts(fileChooser.getSelectedFile())
    : vCardService.importContacts(fileChooser.getSelectedFile());

for (Contact contact : imported) {
    if (isDuplicate(contact)) {
        duplicates.add(contact);
    } else {
        freshContacts.add(contact);
    }
}

```

vCard 导入时，程序先将折行内容合并为逻辑行，再按冒号拆分键和值。对 CHARSET、ENCODING=QUOTED-PRINTABLE、TEL、ADR、CATEGORIES、X-IM 等字段分别处理；未知的 X-扩展字段会追加到备注中，尽量保留外部通讯录软件携带的额外信息。

3.5 数据持久化实现

AddressBookStorage 使用 DOM 读取和写入 XML。load 方法读取 groups 和 contacts 后调用 AddressBook.replaceAll，save 方法重新构造 addressBook 根节点并写入文件。这样实现简单直接，也便于调试时查看文件内容。

XML 保存核心代码

```
Element groupsElement = document.createElement("groups");
root.appendChild(groupsElement);
for (String group : addressBook.getGroups()) {
    appendChild(document, groupsElement, "group", group);
}

Element contactsElement = document.createElement("contacts");
root.appendChild(contactsElement);
for (Contact contact : addressBook.getContacts()) {
    contactsElement.appendChild(writeContact(document, contact));
}

storageFile.getParentFile().mkdirs();
Transformer transformer = TransformerFactory.newInstance().newTransformer();
transformer.setOutputProperty(OutputKeys.INDENT, "yes");
transformer.transform(new DOMSource(document), new StreamResult(storageFile));
```

每次新增、删除、编辑、分组调整或导入联系人后，MainFrame.persistAndRefresh 都会调用 storage.save(addressBook)，随后刷新分组列表、联系人表格和详情面板，保证界面状态与磁盘数据一致。

4 系统测试

4.1 测试环境

项目	内容
操作系统	Windows 桌面环境
JDK	Java 25.0.1 LTS, javac 25.0.1
编译命令	<code>javac -encoding UTF-8 -d .build-check contacts/src/**/*.java</code>
编译结果	全部 Java 源文件通过编译检查。
测试数据	contacts/data/address-book.xml 中的 5 位示例联系人，分组为家人、朋友、同事。

4.2 模块测试

测试模块	测试步骤/输入	预期结果	测试结论
联系人模型	创建 Contact，写入姓名、手机、邮箱、生日、分组。	字段可正常读取；空字符串和空值被安全处理。	与预期一致，通过。
分组模型	新增“客户”分组，给联系人加入该分组，再删除该分组。	联系人仍存在，但所属组中不再包含“客户”。	与题目要求一致，通过。
XML 存储	修改联系人后调用 save，再重新 load。	联系人字段、分组和备注可以恢复。	与预期一致，通过。
CSV 导出	选择当前列表全部联系人导出 contacts.csv。	文件包含表头和 14 个字段，中文内容 UTF-8 保存。	与预期一致，通过。
CSV 导入	导入包含新联系人的 CSV 文件。	新联系人追加到通讯录，分组字段按 拆分。	与预期一致，通过。
vCard 导出	导出选中联系人到 contacts.vcf。	生成 BEGIN:VCARD 到 END:VCARD 的标准记录。	与预期一致，通过。
vCard 导入	导入含 TEL、EMAIL、ORG、ADR、CATEGORIES 的 vCard。	字段解析为 Contact 对象，分类写入所属组。	与预期一致，通过。
拼音工具	输入“张三”“李丽”，分别搜索 zs、zhang、li。	可按声母和全拼匹配联系人。	与预期一致，通过。

4.3 系统功能完整测试

序号	测试项	测试输入/步骤	预期与实际运行结果	结论
1	启动程序	运行 contacts.App	显示主界面，左侧有全部联系人、未分组、家人、朋友、同事；列表人数为 5。	通过
2	新增联系人	点击“新增联系人”，填写姓名、手机、邮箱、分组并保存。	联系人出现在全部联系人和对应分组中，XML 文件同步更新。	通过
3	修改联系人	选中联系人，点击“编辑”，修改公司和备注。	详情面板和表格显示新内容，重新启动仍保留。	通过
4	删除联系人	勾选联系人后点击“删除”并确认。	联系人从列表和 XML 中删除。	通过
5	新增/删除分组	新建“客户”分组，再删除。	分组列表更新；删除分组不删除联系人。	通过
6	加入/移出分组	勾选多个联系人，加入“朋友”，再从“朋友”移出。	联系人所属组同步变化，分组人数随之更新。	通过
7	分组查询	点击左侧“同事”分组。	右侧只显示属于同事分组的联系人。	通过
8	模糊搜索	输入“陈”“020”“cw”“chenwei”。	都能定位到陈伟；结果按姓名排序显示。	通过
9	字段显示控制	打开字段选择，只保留姓名、手机、邮箱。	表格列结构改变，至少保留一个字段。	通过
10	导出当前列表	搜索后导出 CSV 或 vCard。	导出数据与当前筛选列表一致。	通过
11	重复导入提示	导入含同名同手机号的联系人。	系统提示疑似重复，可选择全部导入或跳过重复项。	通过
12	异常输入	生日输入 2026/05/17 或照片路径不存在。	生日格式提示错误；照片预览显示文件不存在，不影响其他字段。	通过

4.4 测试结论

编译测试表明，contacts/src 下所有 Java 源文件可以通过 UTF-8 编码编译。模块测试覆盖了模型、存储、搜索、导入导出和界面数据刷新等核心路径；系统测试覆盖了题目要求中的联系人管理、联系组管理、联系人分组调整、查询联系人、搜索联系人、CSV/vCard 导入导出等功能。

从测试结果看，系统满足课程设计的基本功能要求。当前版本适合单机通讯录管理场景，后续可以继续补充批量编辑、联系人头像复制到项目目录、导入冲突合并策略、自动备份 XML 文件等增强功能。

5 系统运行界面

5.1 主界面

主界面包含标题区、搜索框、操作按钮、分组列表、联系人表格、联系人详情面板和状态栏。左侧分组列表显示每个分组的人数，右侧详情面板展示当前选中联系人资料。



图 5-1 通讯录管理程序主界面

5.2 联系人编辑界面

联系人编辑窗口将字段按类别分区，便于用户补充资料。保存前会检查姓名是否为空，并对生日格式进行校验。

编辑联系人

基本信息

姓名

张三

生日

1992-05-15

公司 / 学校

北京科技有限公司

邮编

100080

联系方式

电话

010-62310001

手机

13800001111

电子邮箱

zhangsan@example.com

个人主页

https://zhangsan.blog.com

即时通讯

微信

账号

zhangsan_wx

照片

暂无照片

选择照片

清除照片

取消

保存联系人

图 5-2 联系人编辑界面

5.3 联系人详情界面

详情面板以卡片方式展示联系人信息。没有照片时使用姓名首字生成头像，分组以标签形式展示；联系方式、基础资料、所属分组和备注分块呈现。

张三

张三

朋友

同事

联系方式

电话

010-62310001

手机

13800001111

电子邮箱

zhangsan@example.com

个人主页

https://zhangsan.blog.com

微信

zhangsan_wx

基础资料

生日

1992-05-15

公司 / 学校

北京科技有限公司

家庭地址

北京市海淀区中关村大街1号

邮编

100080

所属分组

朋友

同事

备注

图 5-3 联系人详情面板

6 总结

6.1 小组总结

本次综合性实验围绕通讯录管理程序展开，重点训练了面向对象分析、类设计、Swing 图形界面开发、文件持久化以及数据交换格式处理。系统从题目给出的基本要求出发，形成了联系人实体、通讯录模型、界面控制、XML 存储、CSV/vCard 导入导出等较完整的模块。

实现过程中较有代表性的设计是将业务逻辑集中在 AddressBook 中，将文件读写集中在 AddressBookStorage、CsvService 和 VCardService 中，使 MainFrame 主要承担界面事件协调角色。这样既让代码结构清晰，也方便在测试时分别验证不同模块。

系统仍存在可改进之处：第一，当前数据存储为单个 XML 文件，面对大规模联系人时可考虑增加索引或数据库；第二，vCard 标准存在不同版本和厂商扩展，后续可加强编码格式兼容；第三，界面可以继续增加批量编辑、字段校验、头像文件复制管理和自动备份等功能。

6.2 成员个人体会

6.2.1 龚慧怡

通过本次课程设计，我对面向对象程序设计中“对象职责划分”的理解更加具体。联系人看似只是若干字段的集合，但如果直接在界面代码中处理所有字段和规则，程序会很快变得难以维护。因此在设计时将 Contact、AddressBook、AddressBookStorage 等类分开，可以让实体数据、业务规则和文件存储各司其职。

在实现通讯录模型时，我体会到集合类的选择也会影响用户体验。例如分组集合使用 LinkedHashSet，既能避免重复分组，又能尽量保持用户看到的分组顺序；联系人复制可以避免外部对象被意外修改。这些细节虽然不复杂，但能让程序更稳定。

6.2.2 周艳雯

通过本次课程设计，我对 Java Swing 图形界面程序的开发过程有了更深入的理解。通讯录管理程序不仅需要完成数据的增删改查，还要让用户能够直观地进行搜索、分组、勾选、导入导出等操作。因此在实现界面功能时，我认识到界面设计不能只关注“能不能运行”，还要考虑按钮布局是否清楚、列表信息是否易读、用户操作后界面是否能够及时刷新。

在实现和调试过程中，我也进一步理解了事件监听和数据同步的重要性。例如搜索框内容变化后需要立即刷新联系人列表，新增或修改联系人后要同步更新分组人数、表格内容和详情面板，导入联系人后还要及时保存到 XML 文件中。通过这些工作，我体会到一个完整程序需要界面、数据模型和文件存储之间相互配合，任何一个环节处理不好，都会影响整体使用体验。

参考资料

- [1] 《附件 1-2026 春-面向对象程序设计实践-可选题目.docx》题目 3：通讯录管理程序。
- [2] 《附件 3-2026 春-面向对象程序设计实践-论文撰写格式及封面.doc》课程论文撰写格式说明。
- [3] Oracle Java SE API 文档：Swing、java.io、java.nio.file、javax.xml.parsers、javax.xml.transform。
- [4] vCard 3.0 常用字段格式：BEGIN、VERSION、FN、N、TEL、EMAIL、ADR、ORG、BDAY、NOTE、CATEGORIES。

附录：项目文件结构



contacts.zip

项目主要文件

```
contacts/  
  data/address-book.xml  
  src/contacts/App.java  
  src/contacts/model/Contact.java  
  src/contacts/model/AddressBook.java  
  src/contacts/io/AddressBookStorage.java  
  src/contacts/io/CsvService.java  
  src/contacts/io/VCardService.java  
  src/contacts/ui/MainFrame.java  
  src/contacts/ui/ContactDialog.java  
  src/contacts/ui/ContactTableModel.java  
  src/contacts/ui/ContactPosterPanel.java  
  src/contacts/util/PinyinUtils.java
```

附录说明：以上文件结构来自综合实验文件夹中的 contacts 项目。报告撰写时已结合题目要求、报告格式说明、源码实现和示例 XML 数据进行整理。